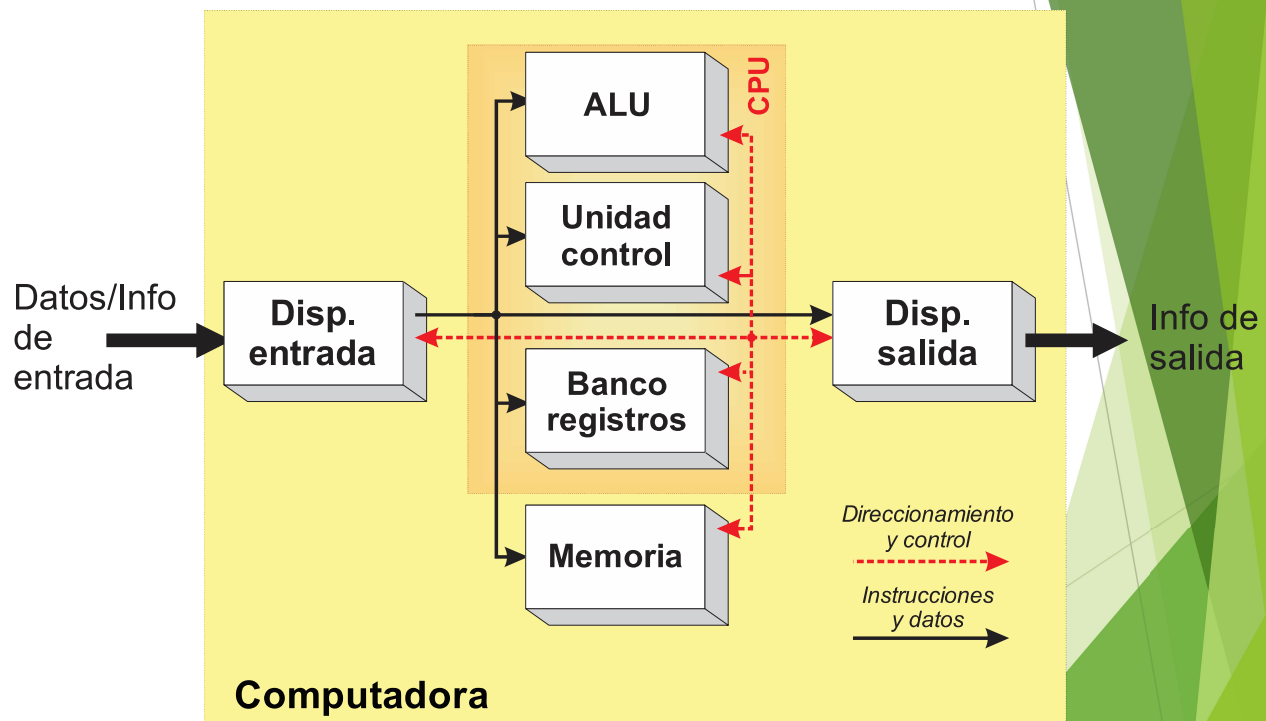


Flujo de Datos



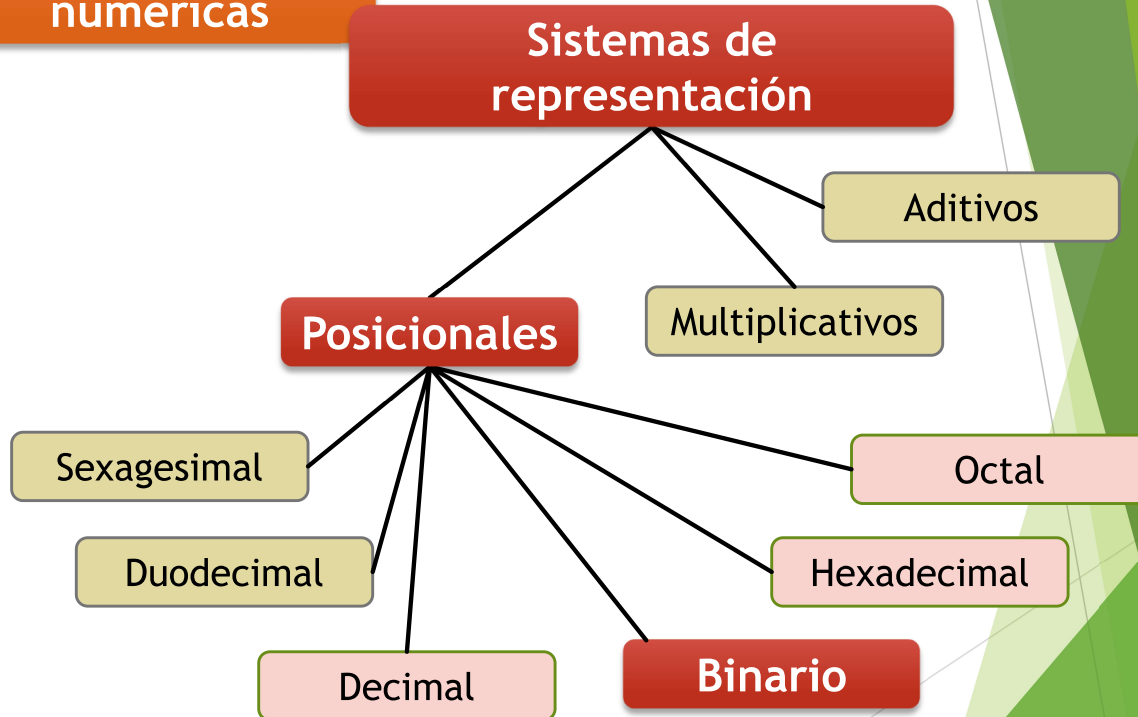
Sistema de Representación Numérica

- Un sistema de representación numérico $\mathcal{N}$ es un sistema de representación formado por un conjunto de símbolos $\mathcal{S}$ y de reglas de generación $\mathcal{R}$ que permiten construir todos los números válidos de determinada estructura algebraica.

$$\mathcal{N} = (\mathcal{S}, \mathcal{R})$$

- Una misma estructura numérica puede tener múltiples sistemas de representación, cambiantes según la época, el lugar, el área de utilización, etc..
- Ej.:** Para los enteros podemos emplear la representación **decimal**, la **binaria**, la **hexadecimal**, la sexagesimal, etc..
- A los racionales podemos optar por expresarlos de manera decimal, como fracciones o cociente de enteros, etc..

Estructuras numéricas



Propiedades de la Potenciación

Producto de potencias de igual base

La multiplicación de potencias de la misma base es otra potencia de la misma base cuyo exponente es la suma de los anteriores

$$A = B^x \cdot B^y \cdot B^z = B^{(x+y+z)} = B^e \quad \text{donde } e = X+Y+Z$$

$$A = 5^4 \cdot 5^2 = \underbrace{5 \cdot 5 \cdot 5 \cdot 5}_4 \cdot \underbrace{5 \cdot 5}_2 = 5^{4+2} = 5^6 = 15625$$

Cociente de potencias de igual base

La división de potencias de la misma base es otra potencia de la misma base cuyo exponente es la diferencia de los anteriores

$$A = B^x / B^y = B^{(x-y)} = B^e \quad \text{donde } e = X-Y$$

$$A = 5^4 / 5^2 = \underbrace{5 \cdot 5 \cdot 5 \cdot 5}_4 / \underbrace{5 \cdot 5}_2 = 5^{4-2} = 5^2 = 25$$

Propiedades de la Potenciación

Potencia 0 de la base

Todo número elevado a la potencia cero dará 1 como resultado

$$A = B^0 = 1 \quad (\text{para todo } B)$$

$$A = 5^0 = 5^{2-2} = \underbrace{5^2}_{2} / \underbrace{5^2}_{2} = \underbrace{5.5}_{2} / \underbrace{5.5}_{2} = 1$$

Potencia 1 de la base

Todo número elevado a la potencia uno dará el valor de la base como resultado

$$A = B^1 = B \quad (\text{para todo } B)$$

$$A = \underbrace{5^1}_1 = 5$$

Propiedades de la Potenciación

Potencia de potencia

Toda potencia de potencia dará es igual otra potencia de la misma base cuyo exponente es el producto de los exponentes anteriores

$$A = (B^x)^y = B^{x \cdot y} = B^z \quad \text{donde } Z=x \cdot y$$

$$A = (5^2)^3 = \underbrace{(5.5)}_2 \cdot \underbrace{(5.5)}_2 \cdot \underbrace{(5.5)}_2 = \underbrace{5.5.5.5.5.5}_3 = 5^{2 \cdot 3} = 5^6 = 15625$$

Potencia de exponentes negativos

Cualquier base elevada a un exponente negativo es igual a la inversa de la misma base elevada a un exponente positivo

$$A = B^{-y} = 1/B^y \quad (\text{para todo } B \text{ distinto de } 0)$$

$$A = 5^{-4} = 5^{0-4} = 5^0 / 5^4 = 1 / 5^4 = 1/625 = 0,0016$$

Sistemas Numéricos

► Sistemas de Notación Posicional (SNP)

- Conjunto de “N” cantidad de símbolos que se pueden combinar entre sí
- Posición: cada dígito tiene un peso diferente según la posición que ocupa
- Base de un SNP: Para una base “B” se tienen “B” símbolos posibles (desde 0 a B-1)
 - Ejemplos: Base 2, tiene 2 símbolos: “0” y “1”
Base 8, tiene 8 símbolos: “0”, “1”, “2”, “3”, “4”, “5”, “6”, “7”
- Expresión de un número en potencias de su base: Para un número “N” con base “B” con “n” dígitos (d) se tiene que:

$$N = d_n \cdot B^n + d_{n-1} \cdot B^{n-1} + \dots + d_0 \cdot B^0$$

Sistemas Numéricos

► Ejemplos. Sistemas de Notación Posicional (SNP)

- Decimal
 - Símbolos {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
- Binario (Lenguaje de computadora)
 - Símbolos {0, 1}
- Octal (Código compacto)
 - Símbolos {0, 1, 2, 3, 4, 5, 6, 7}
- Hexadecimal (Código compacto)
 - Símbolos {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F}
- Otras bases
 - Por ej base 5, base 3, etc que no se utilizan en sistemas computacionales

Sistema Decimal (B=10)

$$B = 10 \rightarrow S_{10} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

$$\mathcal{A} = \mathcal{A}_{n-1} \dots \mathcal{A}_1 \mathcal{A}_0, \mathcal{A}_{-1} \dots \mathcal{A}_{-m} = \sum_{i=-m}^{(n-1)} A_i \cdot 10^i$$

$$A = \overbrace{49}^{n=2}, \overbrace{625}^{m=3} \quad \left\{ \begin{array}{l} A_1=4, A_0=9, A_{-1}=6, A_{-2}=2 \text{ y} \\ A_{-3}=5 \end{array} \right.$$

$$A = \sum_{i=-3}^{(2-1)} A_i \cdot 10^i$$

$$A = 4 \cdot 10^1 + 9 \cdot 10^0 + 6 \cdot 10^{-1} + 2 \cdot 10^{-2} + 5 \cdot 10^{-3}$$

$$A = 4 \cdot 10 + 9 \cdot 1 + 6 \cdot 0,1 + 2 \cdot 0,01 + 5 \cdot 0,001$$

$$A = 40 + 9 + 0,6 + 0,02 + 0,005 = (49,625)_{10}$$

Sistema Binario (B=2)

$$B = 2 \rightarrow S_2 = \{0, 1\}$$

$$\mathcal{A} = \mathcal{A}_{n-1} \dots \mathcal{A}_1 \mathcal{A}_0, \mathcal{A}_{-1} \dots \mathcal{A}_{-m} = \sum_{i=-m}^{(n-1)} A_i \cdot 2^i$$

$$A = \overbrace{101100}^{n=6}, \overbrace{101}^{m=3} \quad \left\{ \begin{array}{l} A_5=1, A_4=0, A_3=1, A_2=1, A_1=0, \\ A_0=0 \\ A_{-1}=1, A_{-2}=0 \text{ y } A_{-3}=1 \end{array} \right.$$

$$A = \sum_{i=-3}^5 A_i \cdot 2^i$$

$$A = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3}$$

$$A = 32 + 8 + 4 + 0,5 + 0,125 = (44,625)_{10}$$

Sistema Hexadecimal (B=16)

$$B = 16 \rightarrow S_{16} = \{0, 1, \dots, E, F\}$$

$$\mathcal{A} = \mathcal{A}_{n-1} \dots \mathcal{A}_1 \mathcal{A}_0, \mathcal{A}_{-1} \dots \mathcal{A}_{-m} = \sum_{i=-3}^5 \mathcal{A}_i \cdot 16^i$$

$$A = \text{B108,E01}$$

$\xleftarrow{n=4}$
 $\xrightarrow{m=3}$

$$\begin{cases} A_3=B=11, A_2=1, A_1=1, A_0=8 \\ A_{-1}=E=14, A_{-2}=0 \text{ y } A_{-3}=1 \end{cases}$$

$$A = \sum_{i=-3}^3 \mathcal{A}_i \cdot 16^i$$

$$A = B \cdot 16^3 + 1 \cdot 16^2 + 0 \cdot 16^1 + 8 \cdot 16^0 + E \cdot 16^{-1} + 0 \cdot 16^{-2} + 1 \cdot 16^{-3}$$

$$A = 11 \cdot 4096 + 1 \cdot 256 + 8 \cdot 1 + 14 \cdot 0,0625 + 1 \cdot 0,0002441$$

$$A = 45056 + 256 + 8 + 0,875 + 0,000244140 \approx 45320,875$$

SRN Posicionales de base B

Digito de la representación	4	...	3	2	1	-1	...	-3
Índice $i = n-1$	3	...	2	1	0	-1	...	-3
Símbolos	$\mathcal{A}_{n-1}$	...	$\mathcal{A}_2$	$\mathcal{A}_1$	$\mathcal{A}_0$	$\mathcal{A}_{-1}$	...	$\mathcal{A}_{-m}$
Peso B^i	B^{n-1}	...	B^2	B^1	1	B^{-1}	...	B^{-m}

$$\mathcal{A}: \mathcal{A}_{n-1} \dots \mathcal{A}_1 \mathcal{A}_0, \mathcal{A}_{-1} \dots \mathcal{A}_{-m}$$

$\xleftarrow{n \text{ dígitos parte entera}}$
 $\xrightarrow{m \text{ dígitos fraccionarios}}$

$$A = \mathcal{A}_{n-1} \cdot B^i + \dots + \mathcal{A}_1 \cdot B^1 + \mathcal{A}_0 \cdot B^0 + \mathcal{A}_{-1} \cdot B^{-1} + \mathcal{A}_{-m} \cdot B^{-m}$$

$$A = \sum_{i=-m}^{(n-1)} \mathcal{A}_i \cdot B^i$$

Sistemas Numéricos

► Métodos de conversión de Sistemas Numéricos

- Conversión de otras bases a base 10 (decimal), mediante El Teorema Fundamental de la Numeración. Expresa un número como la suma de potencias de su base

- Utilizando la fórmula presentada anteriormente:

$$N = d_n \cdot B^n + d_{n-1} \cdot B^{n-1} + \dots + d_0 \cdot B^0$$

- Ver ejemplos en el pizarrón...

Símbolos de un sistema de base B

La base B de un sistema de representación numérica (SRN) posicional indica la cantidad total de símbolos disponibles para realizar la representación

$$S_B = \{s_0, s_1, \dots, s_{B-1}\}$$

Cuando $B=10$ tenemos el sistema decimal:

$$S_{10} = \{0, 1, \dots, 9\}$$

Cuando $B=2$ tenemos el sistema binario:

$$S_2 = \{0, 1\}$$

Cuando $B=16$ tenemos el sistema hexadecimal y así sucesivamente...

$$S_{16} = \{0, 1, \dots, 9, A, B, C, D, E, F\}$$

SRN posicionales de base $\mathcal{B}$

- 1) Cuanto vale i para cada dígito de estos números y cuanto vale n y m para el número?

Número	i	i_{Max}	n	m	$n + m$
126	2,1,0	2	3	0	3
3	0	0	1	0	1
0,234	0,-1,-2,-3	0	1	3	4
32,1	1, 0, -1	1	2	1	3
0,1001	0,-1,-2,-3, -4	0	1	4	5
130657,23	5, 4, 3, 2, 1, 0, -1, -2	5	6	2	8

- 2) ¿Cuántos dígitos se necesitan para escribir estos números?

Número	n
2^{10}	4
10^3	4

Número	n
$6 \cdot 10^3$	4
$10^3 + 10^2 + 6$	4

Conversión binario a decimal ($A_2 \rightarrow A_{10}$)

Aplicación teorema fundamental

Se aplican directamente el teorema fundamental de la numeración.

$$A = \sum_{i=-m}^{(n-1)} A_i \cdot B^i$$

n	2^n	n	2^n	n	2^n
0	1	8	256	16	65,536
1	2	9	512	17	131,072
2	4	10	1,024	18	262,144
3	8	11	2,048	19	524,288
4	16	12	4,096	20	1,048,576
5	32	13	8,192	21	2,097,152
6	64	14	16,384	22	4,194,304
7	128	15	32,768	23	8,388,608

Como en binario los coeficientes A_i solo pueden ser 0 ó 1, el valor decimal resultará de sumar las ponderaciones r^i de aquellos dígitos con coeficientes en 1.

$$(11010)_2 = 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = (26)_{10}$$

Para agilizar la operación conviene memorizar los valores para las primeras potencias de 2 indicadas en la tabla adjunta,

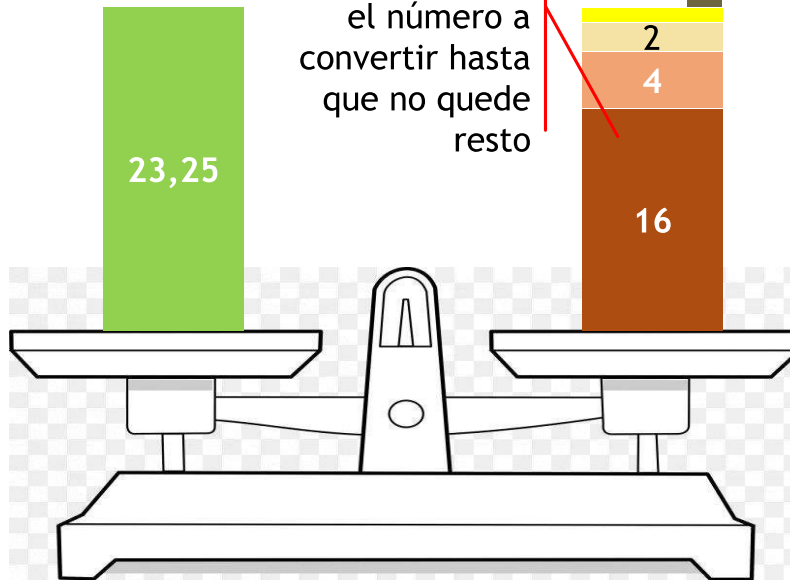
Resta de Potencias de la Base Destino

Decimal a binario

Método 1

Se van restando las potencias de 2 contenidas en el número a convertir hasta que no quede resto

$(10111,01)_2$



n	2^n	2^{-n}
0	1	-- -- -- --
1	2	0.5
2	4	0.25
3	8	0.125
4	16	0.0625
5	32	0.03125
6	64	0.015625
7	128	0.0078125
8	256	0.00390625
9	512	0.001953125
10	1024	0.0009765625
11	2048	0.0004882812
12	4096	0.0002441406
13	8192	0.0001220703

Método 1

Decimal a Binario

Resta de potencias de la base

3127	2^n
2048	2^{11}
1079	
1024	2^{10}
55	
32	2^5
23	
16	2^4
7	
4	2^2
3	
2	2^1
1	
1	2^0

MSD

LSD

Se van restando las potencias de 2 contenidas en el número a convertir hasta que no quede resto.

De manera ordenada de mayor a menor se va formando la representación con el valor que indica la cantidad de veces que estuvo cada potencia.

$$(3127)_{10} = (110000110111)_2$$

n	2^n	2^{-n}
0	1	-- -- -- --
1	2	0.5
2	4	0.25
3	8	0.125
4	16	0.0625
5	32	0.03125
6	64	0.015625
7	128	0.0078125
8	256	0.00390625
9	512	0.001953125
10	1024	0.0009765625
11	2048	0.0004882812
12	4096	0.0002441406
13	8192	0.0001220703

Decimal a Ternario

Resta de potencias de la base

3127,25	3^n
2187,00	$1 \cdot 3^7$
940,25	
729,00	$1 \cdot 3^6$
211,25	
162,00	$2 \cdot 3^4$
49,25	
27,00	$1 \cdot 3^3$
22,25	
18,00	$2 \cdot 3^2$
4,25	
3,00	$1 \cdot 3^1$
1,25	
1,00	$1 \cdot 3^0$
0,25	
0,22	$2 \cdot 3^{-2}$

MSD

LSD

Se van restando la cantidad de veces que está contenida una potencia de 3 en el número a convertir hasta que no quede resto o hasta agotar la cantidad de dígitos fraccionarios pedida.

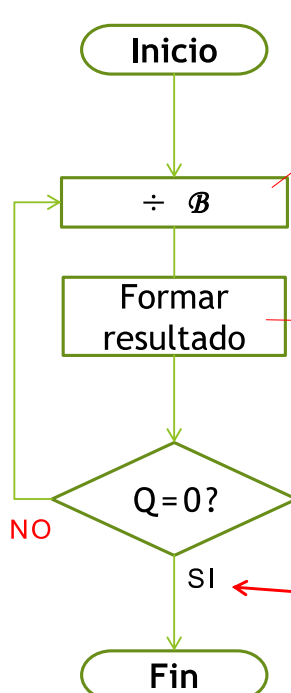
De manera ordenada de mayor a menor se va formando la representación con el valor que indica la cantidad de veces que estuvo cada potencia.

n	3^n	3^{-n}
0	1	...
1	3	0,33333333...
2	9	0,11111111..
3	27	0,03703703...
4	81	0,0123456798...
5	243	...
6	729	...
7	2187	...
8	...	...
9	...	...
10	...	...
11	...	...
12	...	...
13	...	...

$$(3127)_{10} = (11021211,02...)_{3}$$

Decimal a Base B - Parte Entera

Divisiones sucesivas por la base destino



Divido la parte entera del número inicial o el cociente de la pasada anterior por la base B .
Calculo el nuevo cociente entero y el nuevo resto R

Agrego el resto R como dígito a la izquierda (MSB) del resultado BIN

Condición de terminación: $Q_n = 0$

Parte entera del número a convertir: A_{10}

$$A \div B = Q_1 + B_1 \text{ LSB}$$

$$Q_1 \div B = Q_2 + B_2 B_1$$

$$Q_i \div B = Q_{i+1} + B_{i+1} \dots B_2 B_1$$

$$Q_{n-1} \div B = 0 + B_n \dots B_i \dots B_2 B_1$$

Resultado

$$A_{10} = (B_n \dots B_i \dots B_1)_B$$

Decimal a Base B - Parte Entera

Divisiones sucesivas por la base destino

2049	B:2
1024	1
512	0
256	0
128	0
64	0
32	0
16	0
8	0
4	0
2	0
1	0
0	1

LSD
↑
MSD

$$(2049)_{10} = (100000000001)_2$$

3127	B:5
625	2
125	0
25	0
5	0
1	0
0	1

$$(3127)_{10} = (100002)_5$$

3127	B:16
195	7
12	3
0	12 (C)

$$(3127)_{10} = (C37)_{16}$$

3127	B:8
390	7
48	6
6	0
0	6

$$(3127)_{10} = (6067)_8$$

3126	B:3
1042	0
347	1
115	2
38	1
12	2
4	0
1	1

$$(3126)_{10} = (1021210)_3$$

Decimal a Binario - Parte Fraccionaria

Método 2

Multiplicaciones sucesivas por la base

Descompongo el número racional en parte entera y parte fraccionaria

1

$$(3,78125)_{10} = (3)_{10} + (0,78125)_{10}$$

Multiplico por dos solamente las sucesivas partes fraccionarias

2

0,78125	x 2 =	1,5625	Genero el resultado de la conversión comenzando con "0," seguido por los dígitos de las partes enteras de las sucesivas multiplicaciones, ordenadas tal cual se fueron generando (flecha) 3 Condición de terminación: Resto = 0 Representación exacta, sin error
0,5625	x 2 =	1,125	
0,125	x 2 =	0,25	
0,25	x 2 =	0,5	
0,5	x 2 =	01,0	

Reconstruyo el resultado sumando los resultados individuales de convertir parte entera y fraccionaria

5

$$(0,78125)_{10} = (0,11001)_2$$

$$(3,78125)_{10} = (11)_2 + (0,11001)_2 = (11,11001)_2$$

Representación exacta

Decimal a Binario - Parte Fraccionaria

Método 2

Multiplicaciones sucesivas por la base

Descompongo el número racional en parte entera y parte fraccionaria

Multiplico por dos solamente las sucesivas partes fraccionarias

Reconstruyo el resultado sumando los resultados individuales de convertir parte entera y fraccionaria

Genero el resultado de la conversión comenzando con "0," seguido por los dígitos de las partes enteras de las sucesivas multiplicaciones, ordenadas tal cual se fueron generando (flecha)

Condición de terminación: Cantidad máx.. de dígitos significativos alcanzada. La representación tiene error.

Representación con error

MSD

0,

0,81731 x 2 = 1,64462

0,63462 x 2 = 1,26924

0,26924 x 2 = 0,53848

0,53848 x 2 = 1,07696

0,07696 x 2 = 0,15392

(2,81731)₁₀ = (2)₁₀ + (0,81731)₁₀

(0,81731)₁₀ = (0,11010)₂

(2,81731)₁₀ = (10)₂ + (0,11010)₂ = (10,11010)₂

Decimal a base B - Parte Fraccionaria

Método 2

Multiplicaciones sucesivas por la base

	B10 a B2
0,	0,21875
0	0,4375
0	0,875
1	0,75
1	0,5
1	0

MSB
LSD

(0,21875)₁₀ = (0,00111)₂

	B10 a B10
0,	0,824
8	0,24
2	0,4
4	0

(0,824)₁₀ = (0,824)₁₀

	B10 a B2
0,	0,2
0	0,4
0	0,8
1	0,6
1	0,2
0	0,4
0	0,8
1	0,6

	B10 a B2
0,	0,5625
1	0,125
0	0,25
0	0,5
1	0

(0,5625)₁₀ = (0,1001)₂

	B10 a B2
0,	0,1875
0	0,375
0	0,75
1	0,5
1	0

(0,1875)₁₀ = (0,0011)₂

	B10 a B16
0,	0,59375
9	0,5
8	0

(0,59375)₁₀ = (0,98)₁₆

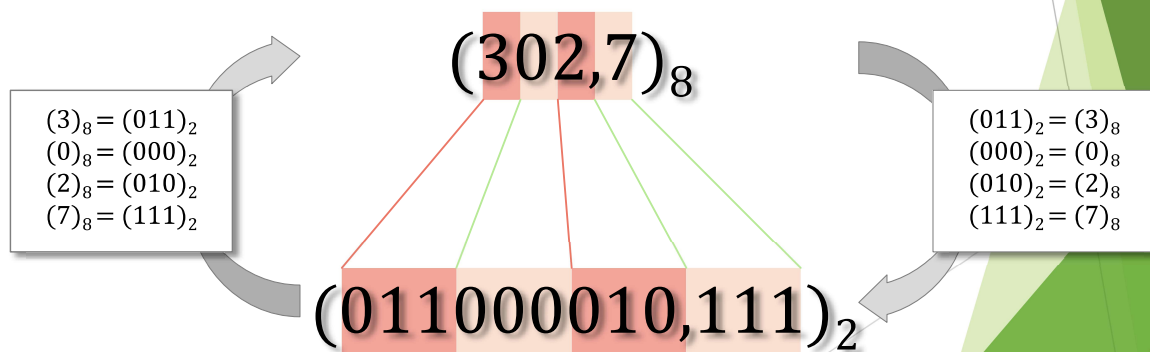
Pasaje Directo entre Binario y Octal

El octal se representa mediante de 8 símbolos:

$$S_8 = \{0, 1, 2, 3, 4, 5, 6, 7\} \quad B = 8 = 2^3$$

Como la base del octal (8) coincide exactamente con una potencia de la base del binario (2^3) cada dígito octal puede ser representado por tres dígitos binarios y viceversa; cada terna de dígitos binarios puede ser reemplazada por un único dígito octal.

Esto permite realizar pasajes directos entre octal y binario por simple sustitución...



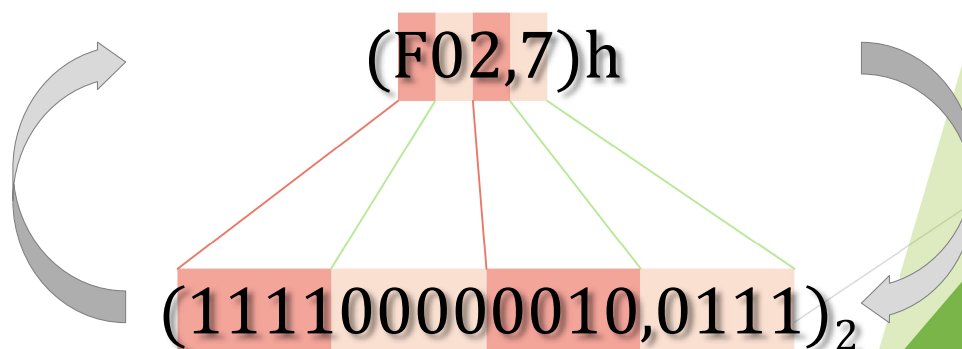
Pasajes entre Binario y Hexadecimal

El hexadecimal se representa mediante de 16 símbolos:

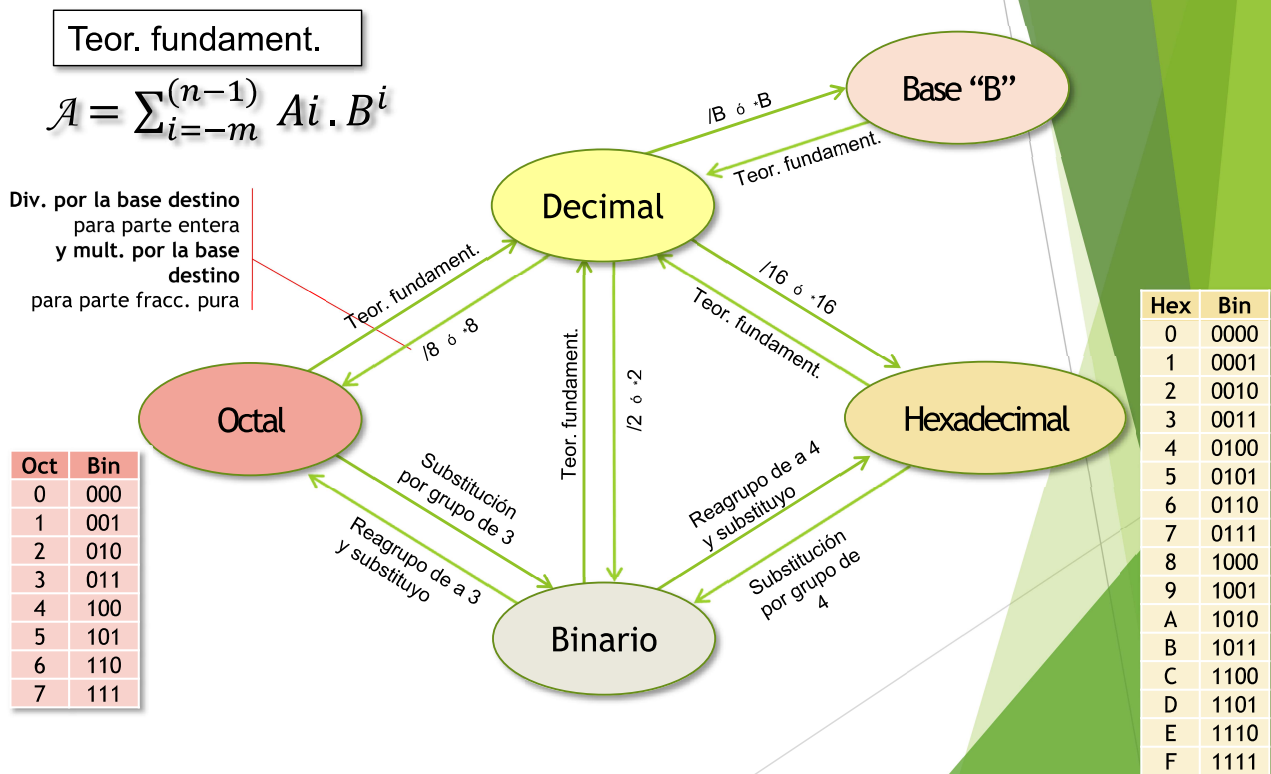
$$S_h = \{0, 1, 2, \dots, E, F\} \quad B = 16 = 2^4$$

Como la base del hexa(16) también coincide exactamente con una potencia de la base del binario (2^4) cada dígito puede ser representado por cuatro dígitos binarios y viceversa; cada cuaterna de dígitos binarios puede ser reemplazada por un único dígito hexa.

Esto permite realizar pasajes directos entre hexa y binario por simple sustitución...



Pasaje entre Todos los SRN



SRN de Ancho Fijo

Al estar limitada la cantidad de dígitos de la representación numérica, automáticamente va a quedar limitado el máximo número que podemos representar. Con representaciones de ancho fijo solo podrán representarse cifras **sin signo** que estén dentro del **rango** entre **0** y dicho **valor máximo**.

Valor mínimo del rango = 0
Valor máximo del rango = $B^n - 1$

La cantidad de números diferentes representables con una cifra de n dígitos en un sistema de base B se llama **capacidad de representación**. Para el caso de números naturales siempre valdrá uno más que el valor máximo del rango sin signo.

$$\text{CapRep} = B^n$$

Base, Ancho y Capacidad de Representación

B = base del sistema de
representación

Dígito binario = bit

$$(A)_2 = (0110\ 1000\ 1101)_2$$

Dígito

$$(A)_{10} = (71.192.000)_{10}$$

n = ancho =
cantidad de dígitos
de la cifra

$$\text{CapRep} = B^n$$

Resumen de Definiciones

Capacidad de representación: *Cantidad de valores o estados representables.*

Rango: *Diferencia entre el mayor y el menor número representables.*

Resolución: *Diferencia entre dos números consecutivos.*

Error absoluto (Ea): *Módulo de la diferencia entre el valor a representar y el valor representado.*

Error absoluto máximo (Ea_{max}): *El de la resolución ó ± la resolución/2*

Error relativo (Er): *Cociente entre el error absoluto y el valor a representar*

Kilo, Mega, Giga y Tera en Binario

n	2 ⁿ	n	2 ⁿ	n	2 ⁿ
0	1	8	256	16	65.535
1	2	9	512	17	131.072
2	4	10	1.024	18	262.144
3	8	11	2.048	19	524.288
4	16	12	4.096	20	1.048.576
5	32	13	8.192	21	2.097.152
6	64	14	16.384	22	4.194.304
7	128	15	32.768	23	8.388.608

Dada la relativa proximidad del valor decimal de ciertas potencias del sistema binario con otras del sistema decimal (por ejemplo 2^{10} con 10^3), se generalizó el uso de los mismos prefijos para ambos sistemas... pero no debemos olvidar a que nos referimos cuando usamos kilo, mega, etc.. en el sistema binario, ya que 1000 no es igual a 1024.

Prefijo	en Decimal	en Binario	Error
Kilo	$10^3 = 1.000$	$2^{10} = 1.024$	24%
Mega	$10^6 = 1.000.000$	$2^{20} = 1.048.576$	49%
Giga	$10^9 = 1.000.000.000$	$2^{30} = 1.073.741.824$	74 %
Tera	$10^{12} = 1.000.000.000.000$	$2^{40} = 1.099.513.162.777$	10%

Binario Sin Signo $(A)_2$; $(A)_{BSS}$

Nomenclatura: $(A)_2$ ó $(A)_{BSS}$

Magnitud representada: Números naturales N con el 0 (números enteros positivos Z^+) dentro del rango.

$$A \in \text{Rango} = \{0, \dots, 2^n - 1\} \cap \text{Rango} \subset Z^+ = \{0, 1, \dots, +\infty\}$$

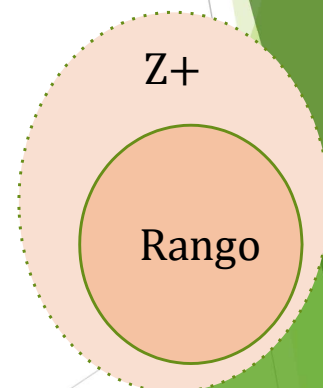
Donde n=ancho en bits de la palabra

$$n \rightarrow 0 \leq A \leq 2^n - 1$$

$$n=8 \rightarrow 0 \leq A \leq 2^8 - 1 = 255$$

$$n=16 \text{ bits} \rightarrow 0 \leq A \leq 2^{16} - 1 = 65.535$$

$$n=32 \text{ bits} \rightarrow 0 \leq A \leq 2^{32} - 1 = 4.294.967.295$$



a_{n-1}	a_{n-2}	...	a_i	...	a_2	a_1	a_0
-----------	-----------	-----	-------	-----	-------	-------	-------

Binario Convenio “Magnitud y Signo” (2MS)

Magnitud representada: Números enteros Z dentro del rango.

$$A \in \text{Rango} = \{-(2^{n-1}-1) \text{ y } (2^{n-1}-1)\} \cap \text{Rango} \subset Z = \{-\infty, \dots, \infty\}$$

Donde n =ancho en bits de la palabra

$$n=8 \text{ bits} \rightarrow -(2^7-1) = -127 \leq A \leq 2^7-1 = 127$$

$$n=16 \text{ bits} \rightarrow -(2^{15}-1) = -32.767 \leq A \leq 2^{15}-1 = 32.767$$

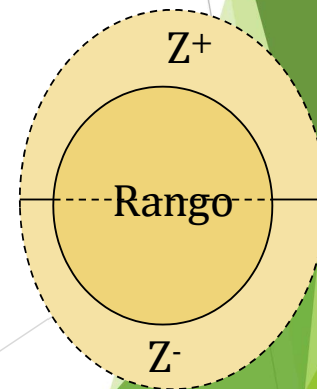
$$n=32 \text{ bits} \rightarrow -(2^{31}-1) = -2.147.483.647 \leq A \leq 2^{31}-1 = 2.147.483.647$$

Ventajas

- Intuitivo: Se parece a la practica habitual con decimales en el mundo real.
- Rango simétrico respecto al cero

Desventajas

- Doble cero
- Desordenado: El orden de la representación no sigue el orden de los números representados



Magnitud y Signo

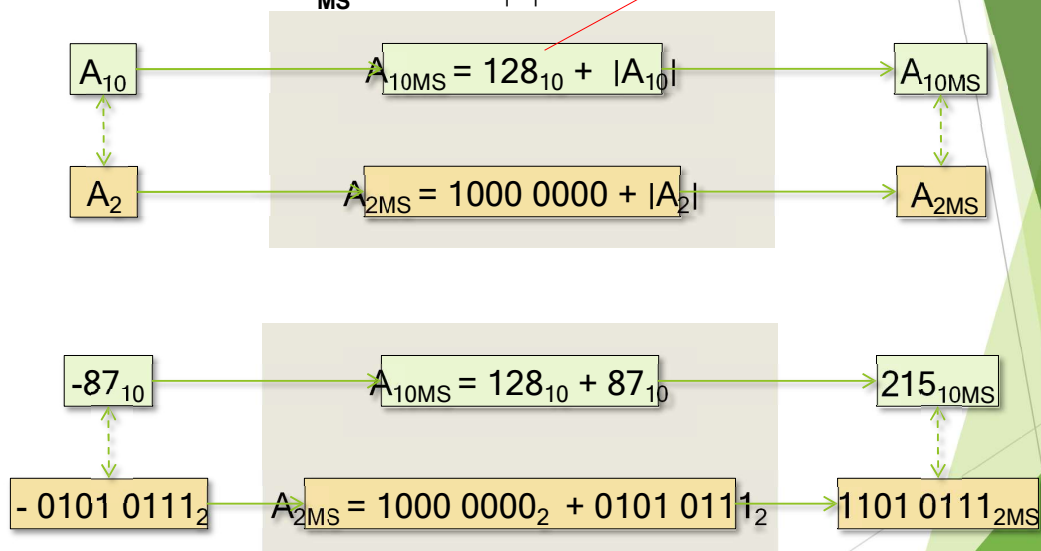
$$A < 0 \wedge n = 8$$

$$A_{MS} = 2^{(n-1)} + |A|$$

$$\text{Si } A < 0 \text{ y } n = 8$$

$$\rightarrow A_{10MS} = 128_{10} + |A_{10}|$$

$$\rightarrow A_{2MS} = 1000\ 0000_2 + |A_2|$$



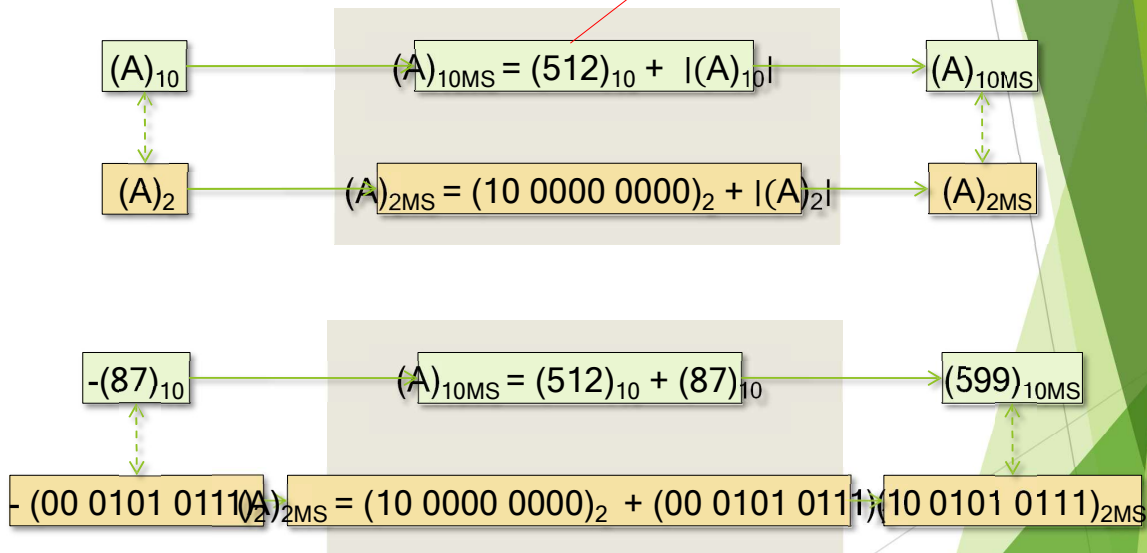
Magnitud y Signo

$$A < 0 \wedge n = 10$$

Si $A < 0$ y $n = 10$

$$\rightarrow (A)_{10MS} = (512)_{10} + |(A)_{10}|$$

$$\rightarrow (A)_{2MS} = (10\ 0000\ 0000)_2 + |(A)_{2}|$$



Convenio "Complemento a Uno": $(A)_{2C1}$

$$C1(A) = 2^n - 1 - |A| \quad \text{sí } n = 8 \rightarrow C1(A) = 255 - |A|$$

Magnitud representada: Números enteros Z dentro del rango.

$$A \in \text{Rango} = \{-(2^{n-1}-1), \dots, (2^{n-1}-1)\} \cap \text{Rango} \subset Z = \{-\infty, \dots, \infty\}$$

Donde n =ancho en bits de la palabra

$$n=8 \text{ bits} \rightarrow -(2^7-1) = -127 \leq A \leq 2^7-1 = 127$$

$$n=16 \text{ bits} \rightarrow -(2^{15}-1) = -32.767 \leq A \leq 2^{15}-1 = 32.767$$

$$n=32 \text{ bits} \rightarrow -(2^{31}-1) = -2.147.483.647 \leq A \leq 2^{31}-1 = 2.147.483.647$$

Ventajas

- Relativa facilidad de suma algebraica implementando el acarreo circular o del último bit (carry around)
- Rango simétrico respecto al cero

Desventajas

- Doble cero
- Desordenado: El orden de la representación no sigue el orden de los números representados

Complemento a Dos

$$C2(A) = 2^n - |A| \quad \text{si } n = 8 \rightarrow C2(A) = 256 - |A| = C1(A) + 1$$

Magnitud representada: Números enteros Z dentro del rango.

$$A \in \text{Rango} = \{-2^{n-1} \text{ y } 2^{n-1}-1\} \cap \text{Rango} \subset \mathbb{Z} = \{-\infty, \dots, \infty\}$$

Donde **n**=ancho en bits de palabra

$$n=8 \text{ bits} \rightarrow -2^7 = -128 \leq A \leq 2^7-1=127$$

$$n=16 \text{ bits} \rightarrow -2^{15} = -32.768 \leq A \leq 2^{15}-1 = 32.767$$

$$n=32 \text{ bits} \rightarrow -2^{31} = -2.147.483.648 \leq A \leq 2^{31}-1 = 2.147.483.647$$

Ventajas

- Gran facilidad para la suma algebraica
- Única representación del cero

Desventajas

- Rango asimétrico respecto al cero
- Desordenado: El orden de la representación no sigue el orden de los números representados

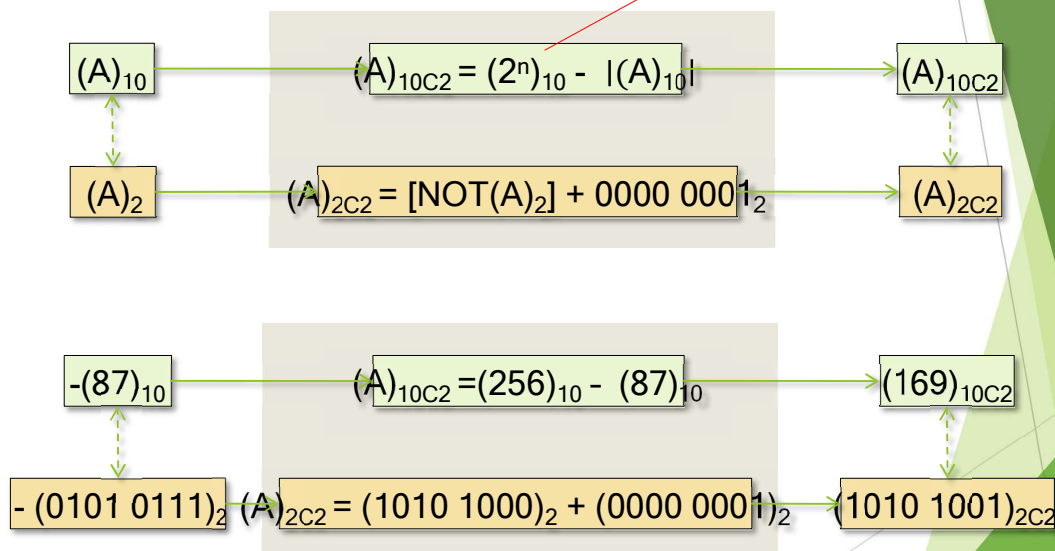
Complemento a Dos

$$A < 0 \wedge n = 8$$

$$\text{Si } A < 0 \text{ y } n = 10$$

$$\rightarrow (A)_{10C2} = 256_{10} - |A_{10}|$$

$$\rightarrow (A)_{2C2} = [\text{NOT}(A)_2] + 1$$



Complemento a Dos

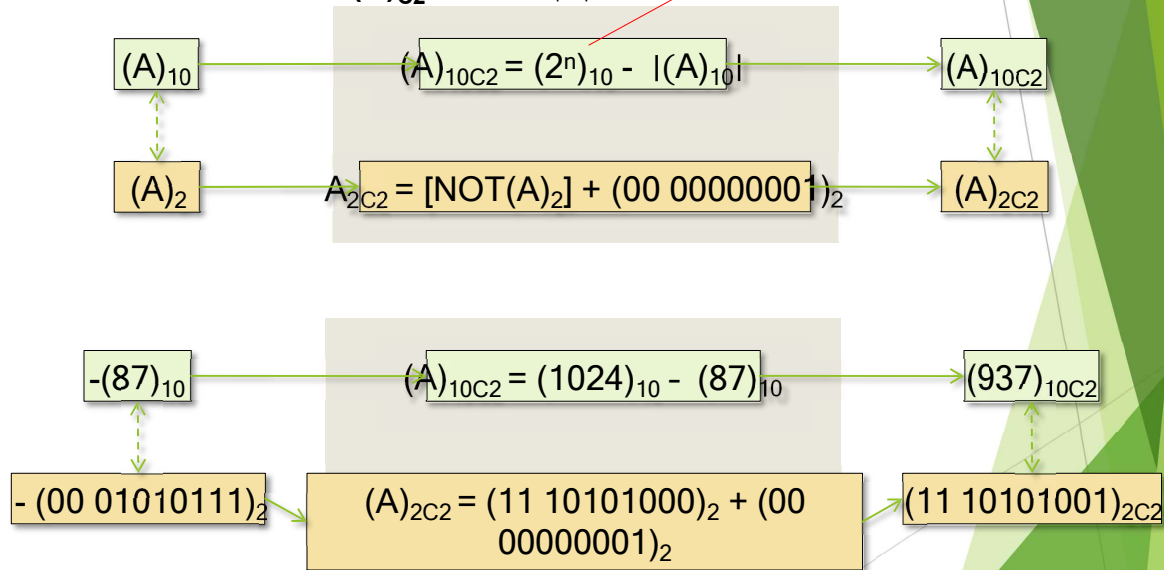
$$A < 0 \wedge n = 10$$

$$(A)_{C2} = 2^n - |A|$$

Si $A < 0$ y $n = 10$

$$\rightarrow (A)_{10C2} = (1024)_{10} - |(A)_{10}|$$

$$\rightarrow (A)_{2C2} = [\text{NOT}(A)_2] + 1$$



Complemento a Dos Rápido

$$A < 0 \wedge n = 10$$

Después del primer uno
INVERTIR

Hasta el primer uno
inclusive
NO INVERTIR

$$(A)_2 = 01\ 0110\ 1000_2$$

$$(A)_{2C2} = 10\ 1001\ 1000_2$$

$$\text{NOT}(A)_2 = 10\ 1001\ 0111_2$$

$$+1 = 00\ 0000\ 0001_2$$

$$(A)_{2C2} = 10\ 1001\ 1000_2$$

=

Exceso $k=2^{n-1}$

$$EK(A) = 2^{n-1} + A \quad \text{sí } n=8 \rightarrow E128(A) = 128 + A$$

Magnitud representada: Números enteros Z dentro del rango.

$$A \in \text{Rango} = \{-2^{n-1} \text{ y } 2^{n-1}-1\} \cap \text{Rango} \subset Z = \{-\infty, \dots, \infty\}$$

Donde n =ancho de palabra)

$$n=8 \text{ bits} \rightarrow -2^7 = -128 \leq A \leq 2^7 - 1 = 127$$

$$n=16 \text{ bits} \rightarrow -2^{15} = -32.768 \leq A \leq 2^{15} - 1 = 32.767$$

$$n=32 \text{ bits} \rightarrow -2^{31} = -2.147.483.648 \leq A \leq 2^{31} - 1 = 2.147.483.647$$

Ventajas

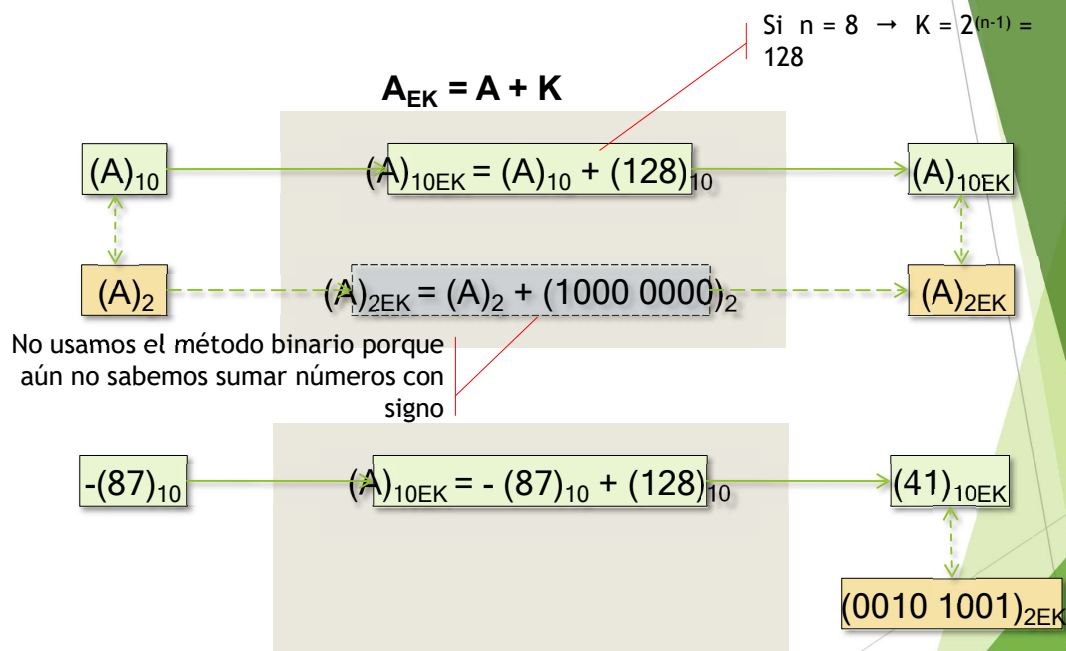
- Ordenado: El orden de la representación sigue el mismo orden de los números representados
- Única representación del cero
- Gran facilidad para la suma algebraica sesgado el MSB para convertirlo en C2

Desventajas

- Rango asimétrico respecto al cero

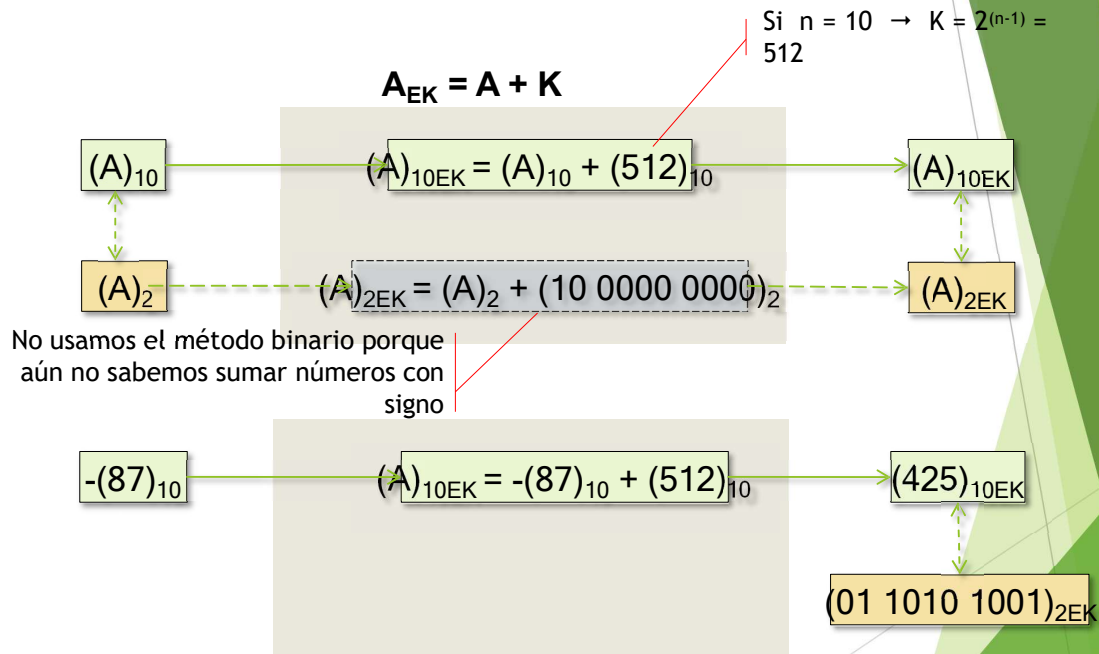
Exceso $K=2^{(n-1)}$

$n = 8$



Exceso $K=2^{(n-1)}$

$n = 10$



Comparación entre Convenios

Para un ancho de palabra de n bits

Convenio de represent.	Doble 0	Capacidad de represent.	Rango	Inverso aditivo	Ordenado
$(A)_{MS}$	Si	2^{n-1}	Simétrico	No	No
$(A)_{2C1}$	Si	2^{n-1}	Simétrico	Casi	No
$(A)_{2C2}$	No	2^n	Asimétrico	Si	No
$(A)_{2EK}$	No	2^n	Asimétrico	Casi	Si

Para un ancho de palabra de 8 bits

Convenio de represent.	Doble 0	Capacidad de represent.	Rango	Inverso aditivo	Ordenado
$(A)_{MS}$	Si	255	-127 a +127	No	No
$(A)_{2C1}$	Si	255	-127 a +127	Casi	No
$(A)_{2C2}$	No	256	-128 a +127	Si	No
$(A)_{2EK}$	No	256	-128 a +127	Casi	Si

Complemento a Dos

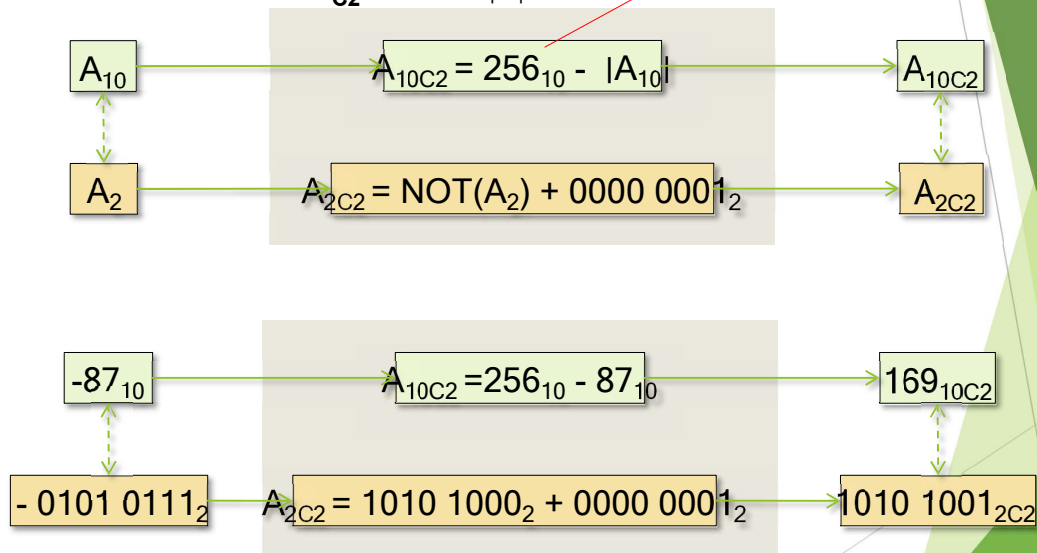
$$A < 0 \wedge n = 8$$

$$A_{C2} = 2^n - |A|$$

Si $A < 0$ y $n = 10$

$$\rightarrow A_{10C2} = 256_{10} - |A_{10}|$$

$$\rightarrow A_{2C2} = \text{NOT}(A_2) + 1$$



Complemento a Dos

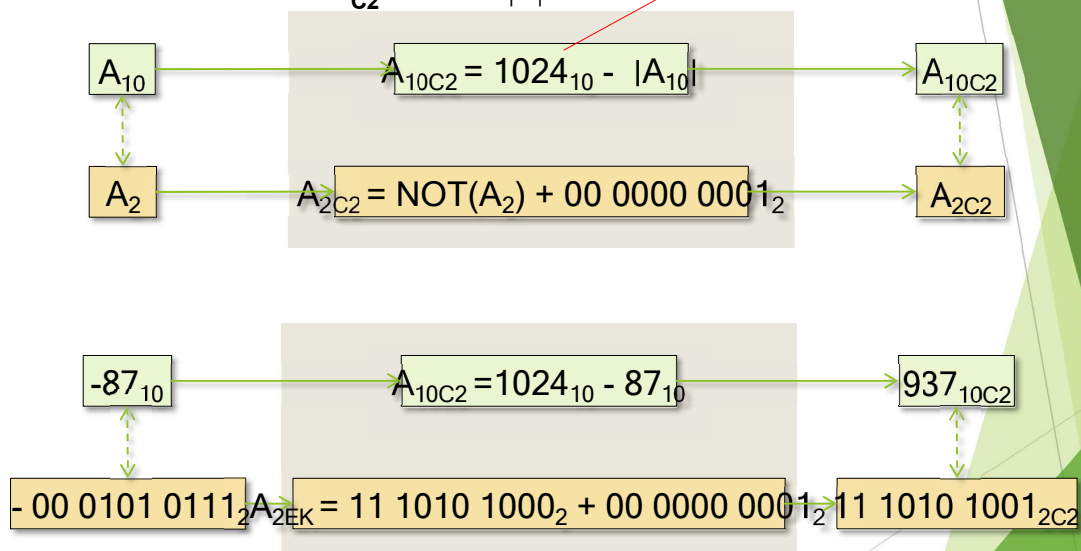
$$A < 0 \wedge n = 10$$

$$A_{C2} = 2^n - |A|$$

Si $A < 0$ y $n = 10$

$$\rightarrow A_{10C2} = 1024_{10} - |A_{10}|$$

$$\rightarrow A_{2C2} = \text{NOT}(A_2) + 1$$



Complemento a Dos

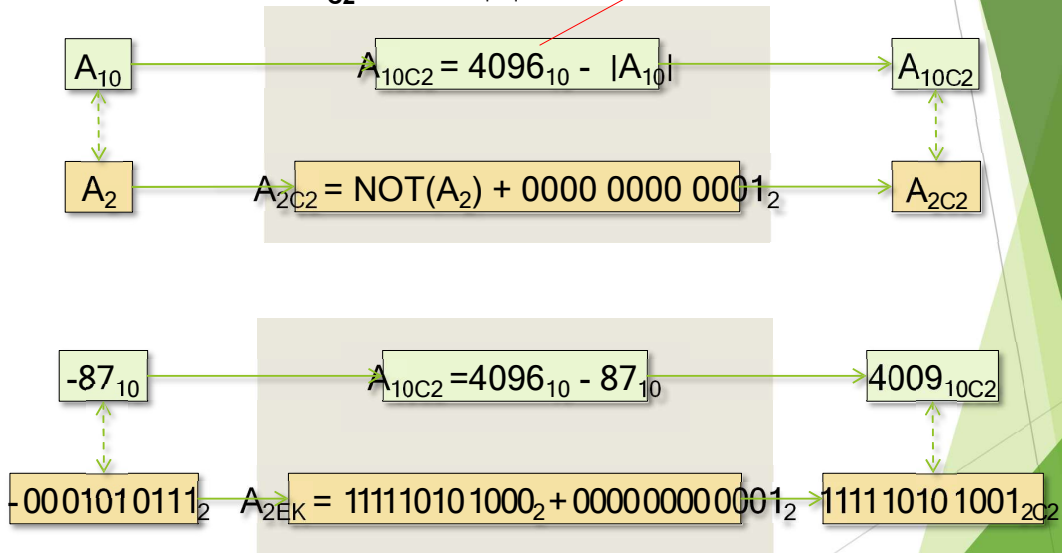
$$A < 0 \wedge n = 12$$

$$A_{C2} = 2^n - |A|$$

Si $A < 0$ y $n = 12$

$$\rightarrow A_{10C2} = 4096_{10} - |A_{10}|$$

$$\rightarrow A_{2C2} = \text{NOT}(A_2) + 1$$



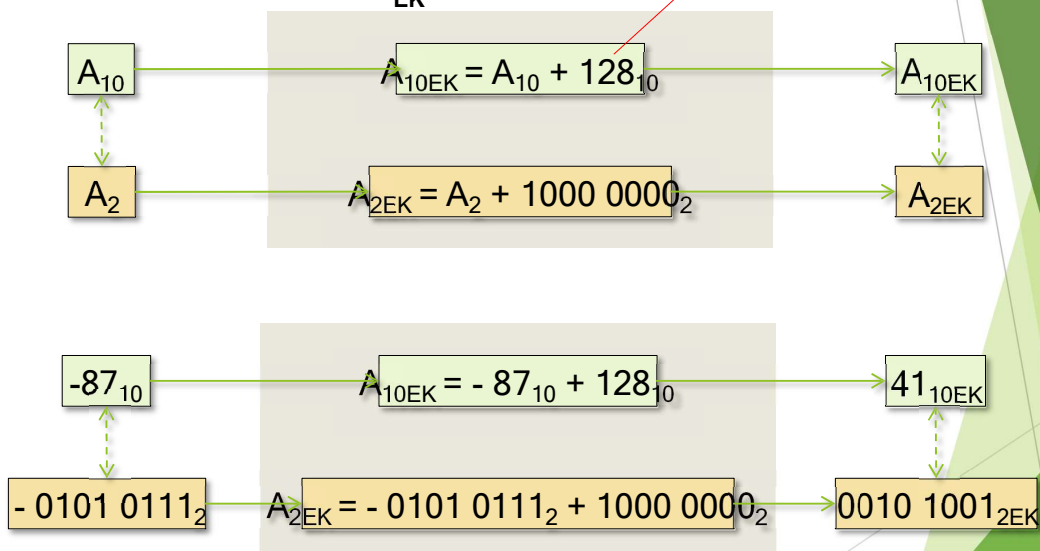
Exceso $k=2^{n-1}$

$$n = 8$$

$$A_{EK} = A + K$$

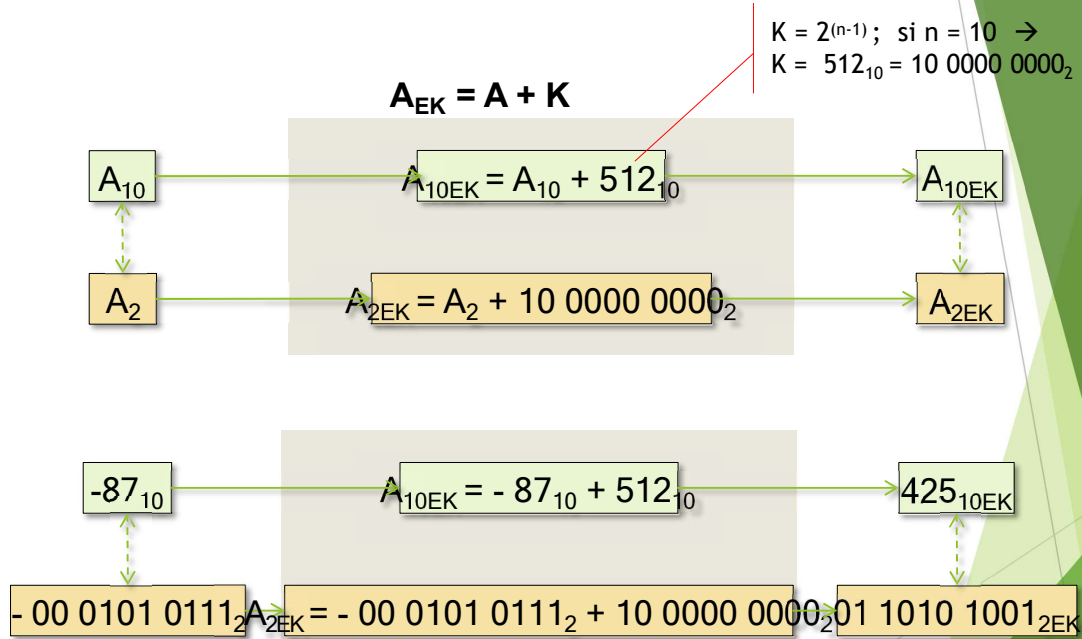
$K = 2^{(n-1)}$; si $n = 8$

$$\rightarrow K = 128_{10} = 1000\ 0000_2$$



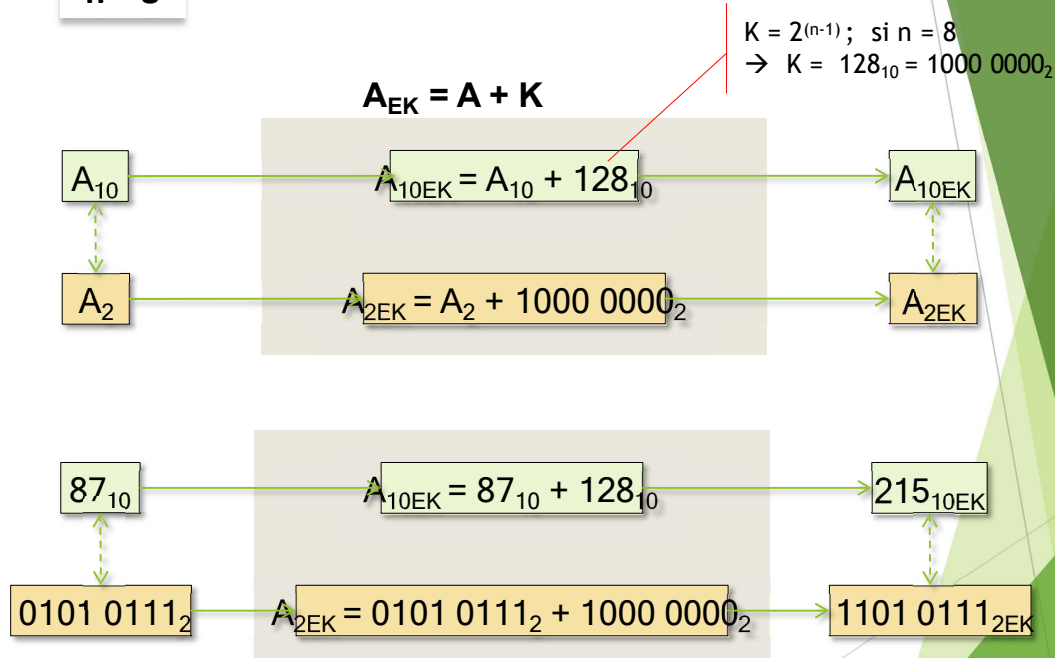
Exceso $k=2^{n-1}$

$n = 10$



Exceso $k=2^{n-1}$

$n = 8$

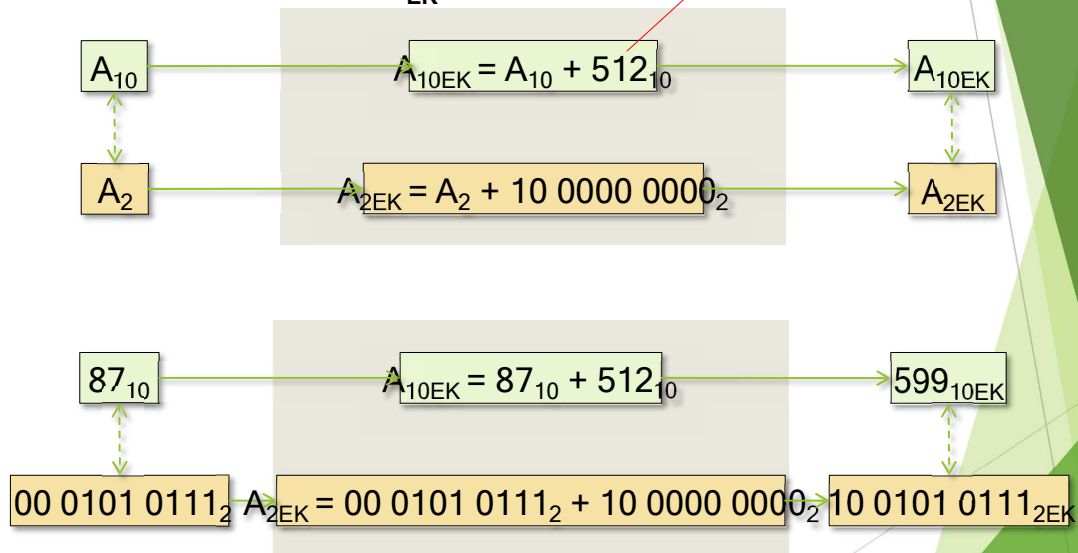


Exceso $k=2^{n-1}$

$n = 10$

$$A_{EK} = A + K$$

$K = 2^{(n-1)}$; si $n = 10 \rightarrow$
 $K = 512_{10} = 10\ 0000\ 0000_2$



Códigos ASCII Alfanuméricos

ASCII “American Standard Code for Information Interchange” (pronunciado 'aski')

Un estándar de representación de caracteres y códigos de control de 7 bits, muy difundido y aún en uso. El 8 bit o se **coloca en cero**, o se le asignan funciones de detección de errores o de codificación extendida (no previstos en este estándar)

		$B_7 B_6 B_5$						
$B_4 B_3 B_2 B_1$	000	001	010	011	100	101	110	111
0000	NULL	DLE	SP	0	@	P	~	P
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	=	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

Carácter alfanumérico:

“a”

Código ASCII: $(0110$

$0001)_2$

Código ASCII: $(61)_{16}$

Código de control

Line feed (avance de línea)

$= (000\ 1010)_2 = (0A)_{16} = (10)_{10}$

Código de control

Carriage return (ret. de

carro)

$= (000\ 1101)_2 = (0D)_{16} = (13)_{10}$

Código ASCII Extendido

ASCII Extendido (8 bits)

BITS DE ZONA: Los primeros 4 bits de mayor significación en cada combinación se denominan bits de **zona** y en la tabla de doble entrada **numeran las columnas 0 a 7**.

BITS DE DÍGITO: Los últimos 4 bits (3, 2, 1 y 0) en cada combinación se denominan bits de **dígito** y en la **tabla numeran las filas de 0 a 15, o de 0 a F en hexadecimal**.

Representación Interna de los Datos en una PC

- La fórmula $\pm M \cdot B^{\pm P}$

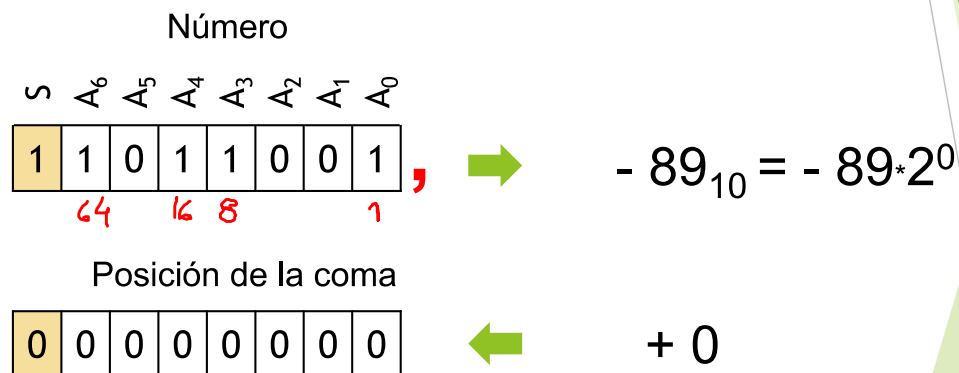
Representa el criterio utilizado para todo sistema posicional, donde:

- “M” es la mantisa, que podrá tratarse como fracción pura o como entero, según se suponga la coma a izquierda o a derecha de “M”.
- “B” es la base del sistema
- “P” es el exponente que indica la posición del punto en “M” los signos “+” y “-” que la anteceden indican su desplazamiento a derecha o izquierda respecto del origen.
- En la memoria sólo se almacenarán la mantisa “M” y la potencia “P” en binario.
- La base “B” y el origen del punto siempre son determinados con anterioridad por el convenio y conocidos por los programas que utilizan esta representación

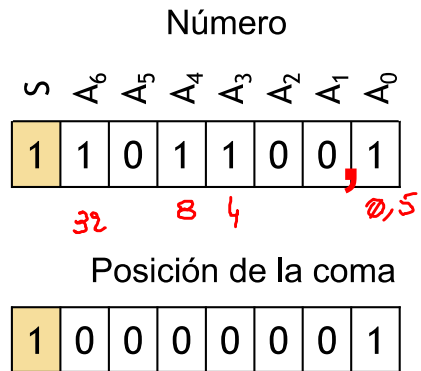
Representación Interna de los Datos en una PC

- Cuando el resultado de una operación supera los límites mayores definidos por el rango, entonces es incorrecto.
- El error se conoce como **overflow de resultado** y actualiza un bit (*flag de overflow*) en un registro asociado a la ALU, conocido como registro de estado o *status register*
- Cuando el resultado de una operación cae en el hueco definido por los límites inferiores del rango (del que se excluye el cero), esto es $0 > x > 0,1$ o $0 < x < 0,1$ es incorrecto.
- El error se conoce como **underflow de resultado**

Punto Fijo



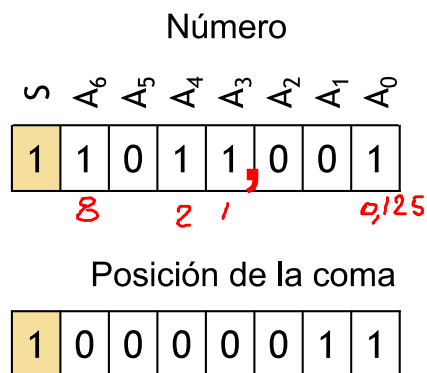
Punto Fijo



$$\rightarrow -44,5_{10} = -89 \cdot 2^{-1}$$

$$\leftarrow -1$$

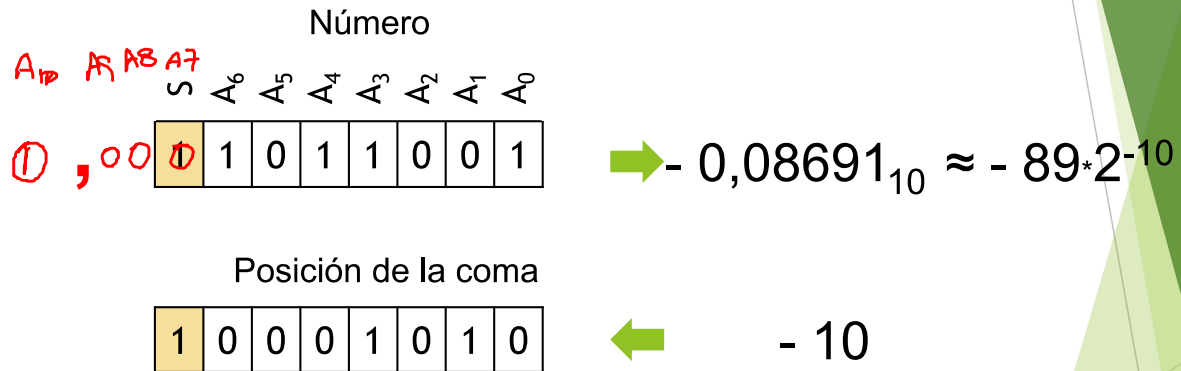
Punto Fijo



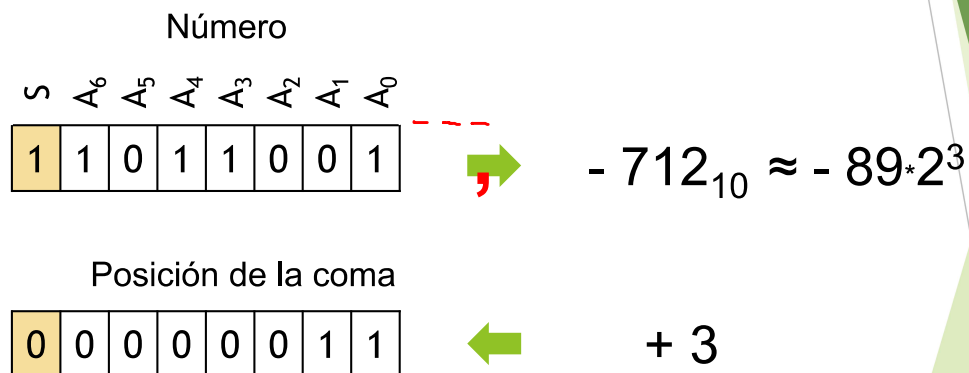
$$\rightarrow -11,125_{10} = -89 \cdot 2^{-3}$$

$$\leftarrow -3$$

Punto Fijo



Punto Fijo



Punto Fijo = Notación Científica

Punto fijo: Un entero, con o sin signo, multiplicado por un factor de escala.

$$\text{Punto fijo} = \text{Entero} * \text{Factor de escala} = Z \cdot 2^x$$

El factor de escala y por lo tanto la posición de la coma solo la conoce el programador/algorithm o el interprete a través del tipo de la variable elegida.

La cantidad de dígitos enteros (n) y fraccionarios (m) de la representación suele indicarse como Qn,m. Por el ancho de palabra del sistema puede inferirse si existe un bit de signo.

Punto Fijo = Notación Científica



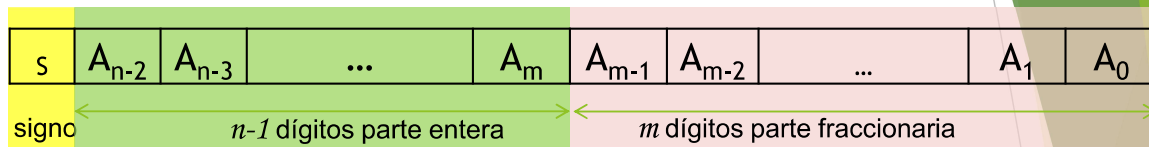
$t = s + n + m$ (s=0 sin signo y s= 1 con signo)

t es constante → mayor rango (mayor n) ↔ menor resolución/exactitud (menor m)

Ventajas:

- Operaciones de suma y resta de operandos con igual factor de escala se realizan exactamente igual que las operaciones entre enteros.
- Error absoluto constante para todo el rango.

Formato Q del punto fijo



Q1.0.15

Sistema de 16 bits → Con signo, 0 dígitos enteros, 15 fraccionarios

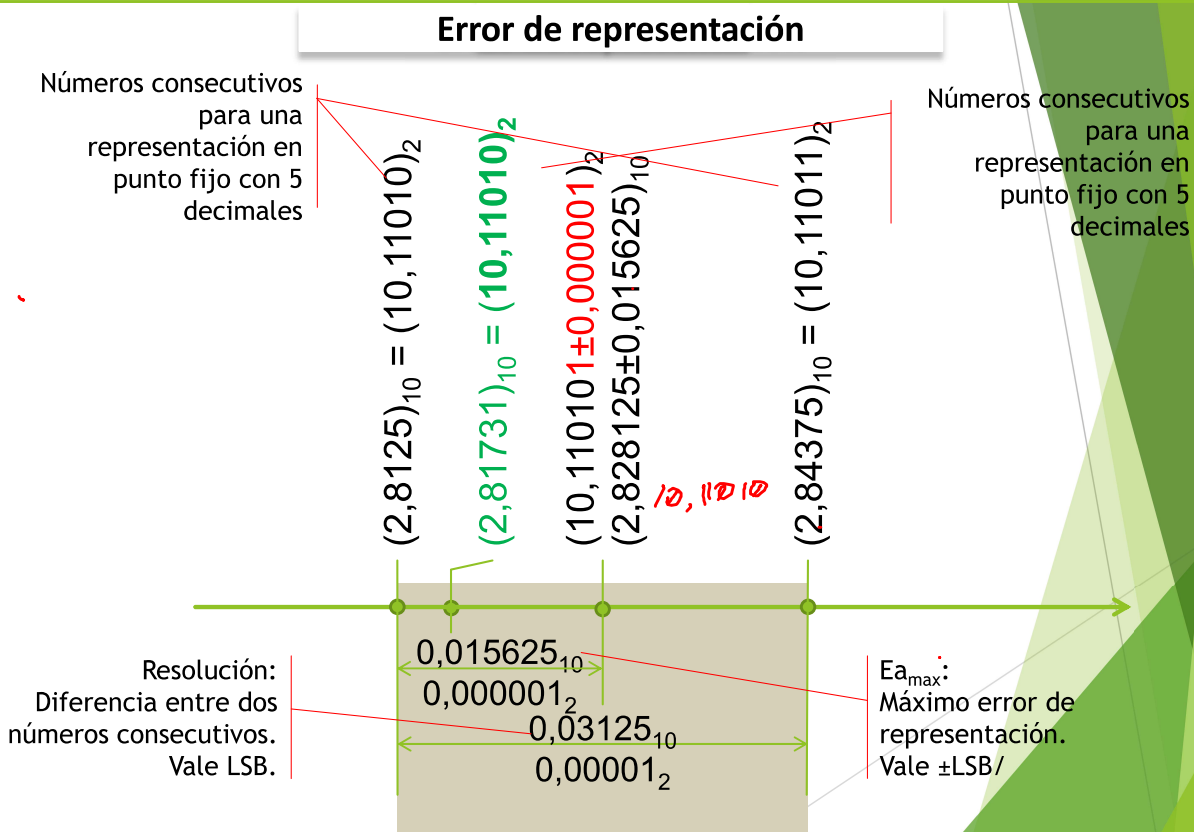
Q0.16.0

Sistema de 16 bits → Sin signo, 16 dígitos enteros, + 0 fraccionarios

Q 1.7.8

Sistema de 16 bits → Con signo, 7 dígitos enteros + 8 fraccionarios

Decimal a Binario - Parte Fraccionaria



Representación Interna de los Datos en una PC

Convenios de representación en punto flotante con exponente en exceso

- Para eliminar el signo del exponente se utilizan convenios que agregan al exponente un exceso igual a 2^{p-1} .
- Esta entidad nueva se denomina característica y es igual a:

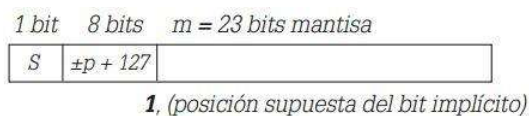
$$\text{CARACTERÍSTICA} = \underbrace{\boxed{\pm P}}_{\text{EXPONENTE}} + \underbrace{\boxed{2^{p-1}}}_{\text{EXCESO}}$$

- Se representa en la forma de signo y magnitud, o sea, **no se utiliza el complemento para los negativos**.

Representación Interna de los Datos en una PC

Punto flotante “exceso 127” (o IEEE p754, simple precisión)

- En esta convención el número se representa en formato de 4 octetos:



- El cálculo de la característica o el exponente responde al criterio empleado en la convención anterior:

$$\pm P + (2^{p-1} - 1) = \pm P + (2^7 - 1) = \pm P + 127_{(10)}$$

Mantisa: tiene la forma “1,” pero el 1 no se almacena.

$$1, \underbrace{b_{22}b_{21}b_{20} \dots b_3b_2b_1b_0}_{23 \text{ bits de mantisa}}$$

Representación Interna de los Datos en una PC

Punto flotante "exceso 127" (o IEEE p754, simple precisión)

Ejemplo 1

4^{to} Rellenar el formato.

Si ahora representamos $+3,5_{(10)} = +11,1_{(2)} = +1,11_{(2)} \cdot 10_{(2)}^{+1(2)}$ calculamos la característica como $(+1) + 01111111 = 10000000$

El signo es positivo, por lo tanto, el formato queda:

pasos

1^{ro} pasar a binario

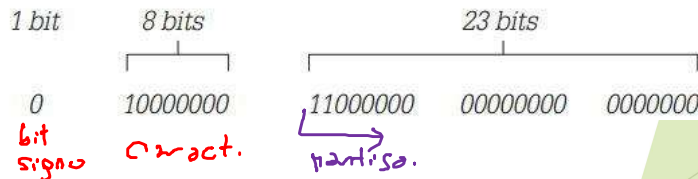
$(11,1)_2$

2^{do} Normalizar = ?

$(1,11) \times (10)^{+1}$

3^{ro} cálculo caract.

$127 + 1 = 128 \rightarrow$ binario



- Cuando corra la coma. si corra a la izquierda el exponente queda con signo (+)
- Si corra a la derecha queda (-)

- Número del exponente = 25
cantidad de lugares que corrió la coma.

Representación Interna de los Datos en una PC

Punto flotante "exceso 127" (o IEEE p754, simple precisión)

Ejemplos 2

El seguimiento de otros ejemplos facilitará su entendimiento:

$$+4,0_{(10)} = +100_{(2)} = 1,0_{(2)} \cdot 10_{(2)}^{+10(2)}$$

0 10000000 00000000 00000000 00000000

El exponente $+10_{(2)}$ significa "desplazar la coma a la derecha dos lugares".

$$+16,0_{(10)} = +10000_{(2)} = 1,0_{(2)} \cdot 10_{(2)}^{+100(2)}$$

0 10000011 00000000 00000000 00000000

El exponente $+100$ significa "desplazar la coma a la derecha cuatro lugares".

$$-1,0_{(10)} = -1_{(2)} = -1,0_{(2)} \cdot 10_{(2)}^{0(2)}$$

1 01111111 00000000 00000000 00000000

Punto flotante “exceso 127” (o IEEE p754, simple precisión)

- Desplazamiento de la coma. Aspectos a tener en cuenta:
 - Si para normalizar la mantisa a la forma “1,” tuve que desplazar la coma a la derecha, entonces el exponente es negativo y el número corresponde a la cantidad de espacios desplazados.
 - Si para normalizar la mantisa a la forma “1,” tuve que desplazar la coma a la izquierda, entonces el exponente es positivo y el número corresponde a la cantidad de espacios desplazados.
 - Si para normalizar la mantisa a la forma “1,” no fue necesario desplazar la coma, entonces el exponente es “0”

Representación Interna de los Datos en una PC

Punto flotante “exceso 127” (o IEEE p754, simple precisión)

- Pasos para representar un número en este formato:
 - 1ro: Convertir el número a binario
 - 2do: Normalizar a la forma “1.”
 - 3ro: Calcular la característica en base al exponente obtenido en el paso anterior
 - 4to: Completar el formato de este convenio (1bit de signo; 8 bits de característica; 23 bits de mantisa, donde solo se guardan los bits que están luego de “1.”, en caso de ser necesario se rellena con “0”)

Representación Interna de los Datos en una PC

Punto flotante “exceso 127” (o IEEE p754, simple precisión)

- Ejemplo con un número periódico:

Ahora considérese el ejemplo siguiente:

$$-0,2_{10} = -0,0011_2 = -1,10011 \cdot 10_2^{-11_2}$$

Esta conversión no es exacta, o sea que la fracción binaria hallada es periódica y, por lo tanto, se aproxima a la fracción decimal original.

El error varía en relación con la cantidad de períodos que se consideren. Cuando se normaliza, los ceros a derecha del punto desaparecen, se indica su existencia con el valor del exponente -11_2 (tres lugares a izquierda del origen de la coma). El número representado queda así:

1 01111100 10011001 10011001 1001100 truncado
mantisa 110011001...

En la mantisa se repite el período tantas veces como el formato lo permite hasta que se trunca.

Representación Interna de los Datos en una PC

Punto flotante “exceso 1023” (o IEEE doble precisión)

- En esta convención el número se representa en formato de 8 octetos, es decir 64 bits.
- Es igual a la convención anterior pero con el doble de bits.
- Utiliza 52 bits de mantisa
- 11 bits para la característica
- 1 bit de signo
- Exponente expresado en exceso 1023



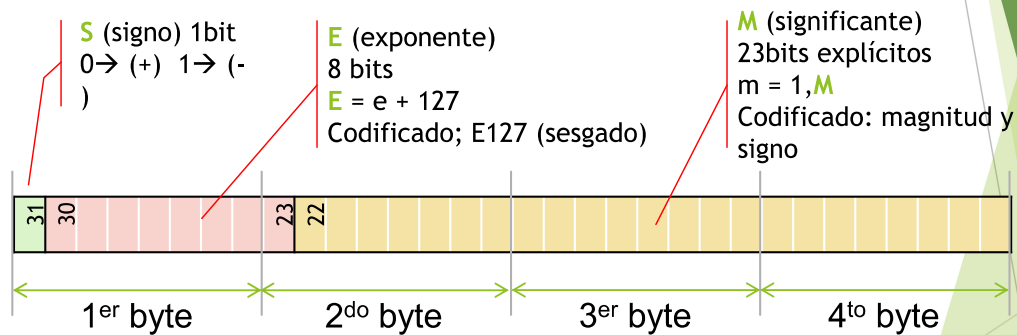
Signo (1bit)
implícito, no se almacena)

1, (bit de mantisa

Punto Flotante Simple Precisión

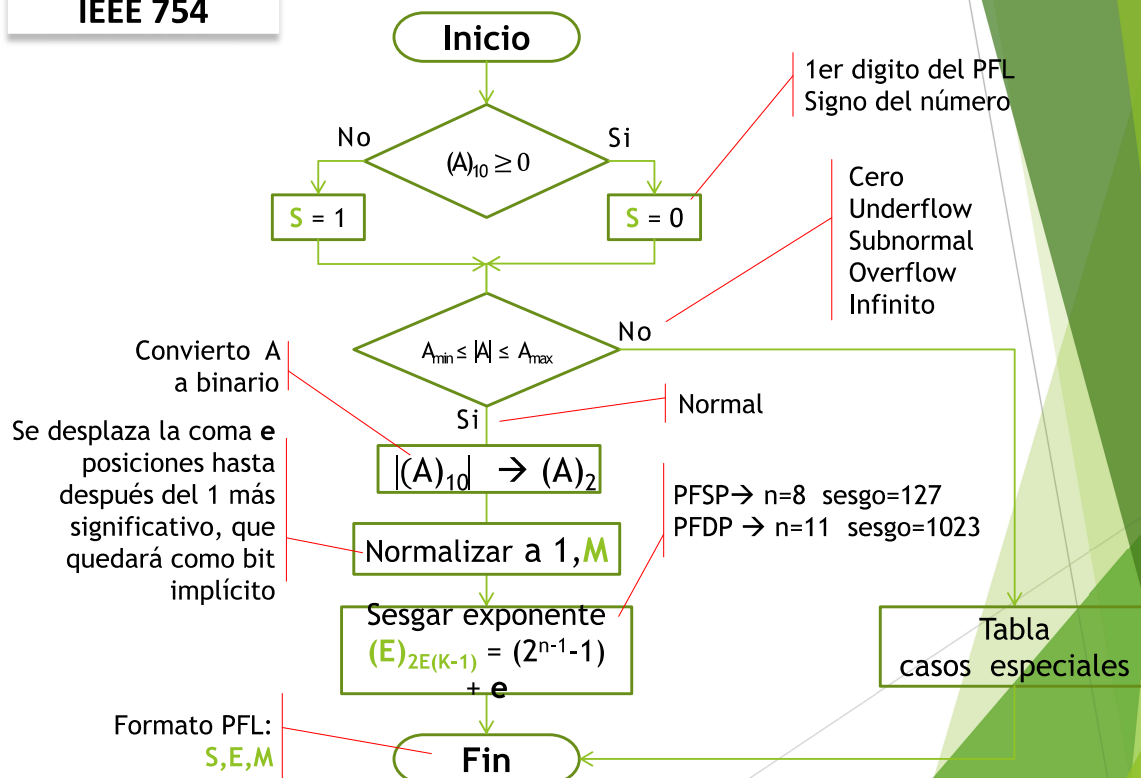
IEEE 754

El formato de punto flotante de simple precisión o binary32 (IEEE754) representa un número decimal $(A)_{10}$ empleando 32 bits (4 bytes) agrupados en tres campos:



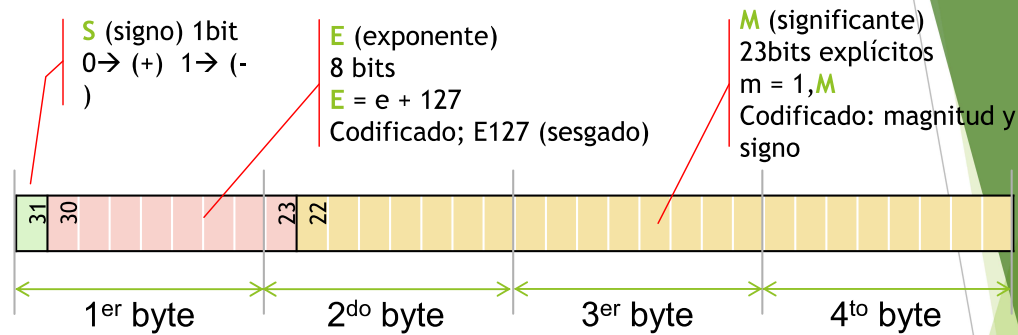
PF (Fijo) a PFL (Flotante)

IEEE 754

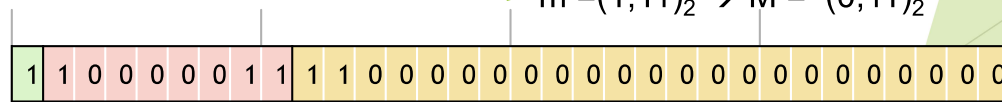


PF a Punto Fl. Simple Precisión (PFSP)

IEEE 754



$s: - \rightarrow S = 1$
 $A = -(28)_{10} = -(11100)_2 = -m \cdot 2^e = -1,11 \cdot 2^4 \rightarrow e = 4 \rightarrow E = 127 + 4 = 131 = (1000\ 0011)_2$
 $m = (1,11)_2 \rightarrow M = (0,11)_2$

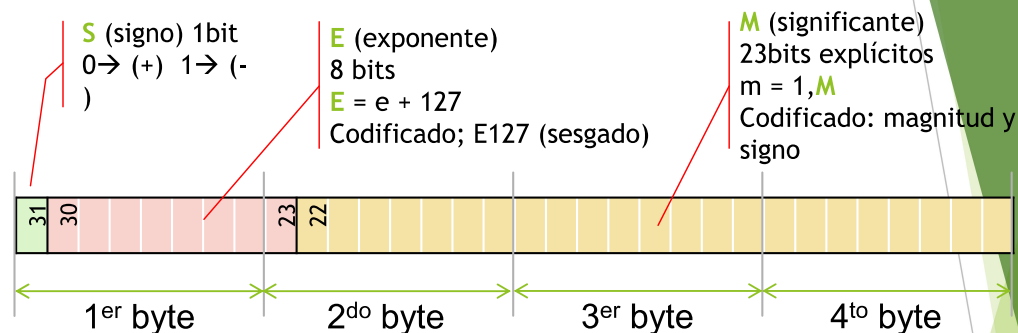


$$(A)_{2PFSP} = 1100\ 0001\ 1110\ 0000\ 0000\ 0000\ 0000\ 0000$$

$$(A)_{16PFSP} = (C1E00000)_{16} = 0xC1E00000$$

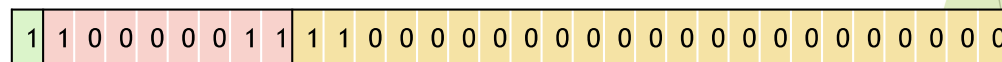
Punto fl. Simple Precisión (PFSP) a PF

IEEE 754



$$(A)_{16PFSP} = (C1E00000)_{16} = 0xC1E00000$$

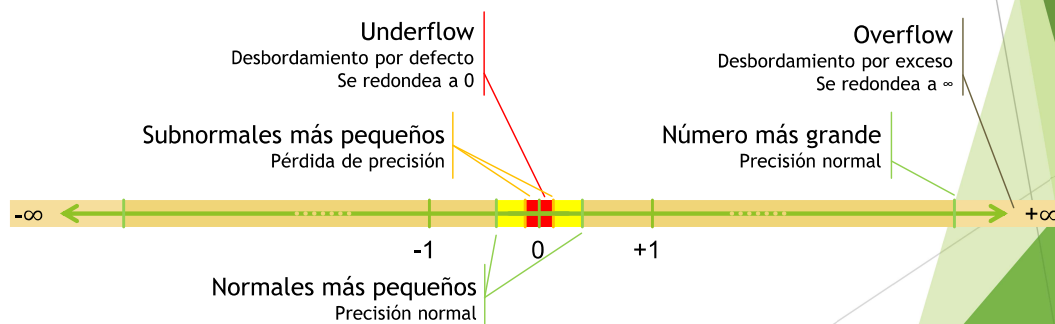
$$(A)_{2PFSP} = 1\ 10000011\ 110\ 0000\ 0000\ 0000\ 0000\ 0000$$



$S = 1 \rightarrow s = "-"$
 $E = (1000\ 0011)_2 = (131)_{10} \rightarrow e = E - 127 = 131 - 127 = 4$
 $A = -1,11 \cdot 2^4 = -m \cdot 2^e = -(11100)_2 = -(28)_{10}$
 $M = (0,11)_2 \rightarrow m = (1,11)_2$

IEEE 754

Formato	Bits Tot	Bits <i>M</i>	Bits <i>S</i>	Bits <i>E</i>	Nº subnorm. más pequeño	Número norm. más pequeño	Número más grande
Precisión simple	32	23	1	8	$\pm \sim 1,4 \cdot 10^{-45}$	$\pm \sim 1,2 \cdot 10^{-38}$	$\pm \sim 3,4 \cdot 10^{38}$
Precisión doble	64	52	1	11	$\pm \sim 5 \cdot 10^{-324}$	$\pm \sim 2,2 \cdot 10^{-308}$	$\pm \sim 1,8 \cdot 10^{308}$



PFL a Punto Fijo

IEEE 754

Tabla para la interpretación de las componentes del punto flotante

Signo <i>S</i>	Exponente <i>E</i>	Significante <i>M</i>	Valor decimal
0	0	0	+ 0
1			- 0
X	0	$\neq 0$	Subnormal
X	$0 < E < 2^n - 1$	X	Normal
0	$2^n - 1$	0	+ ∞
1			- ∞
X	$2^n - 1$	$\neq 0$	NaN

Pasaje de PFL normal a decimal

$$(A)_{10} = (-1)^S * 1, M * 2^{(E-\text{sesgo})}$$

Pasaje de PFL subnormal a decimal

$$(A)_{10} = (-1)^S * 0, M * 2^{(1-\text{sesgo})}$$

Contar y Sumar

Orden creciente

Cuenta ascendente			
r=2	r=10	r=16	r=8
00000	00	00	00
00001	01	01	01
00010	02	02	02
00011	03	03	03
00100	04	04	04
00101	05	05	05
00110	06	06	06
00111	07	07	07
01000	08	08	10
01001	09	09	11
01010	10	0A	12
01011	11	0B	13
01100	12	0C	14
01101	13	0D	15
01110	14	0E	16
01111	15	0F	17
10000	16	10	20

Prop. asociativa de la suma

$$S = (C_i + A) + B = C_i + (A + B)$$

Prop. Conmutativa de la suma

$$S = (A + C_i) + B = B + (C_i + A)$$

Suma binaria				
Ci	A	B	Co	Suma
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Suma Algebraica

Se suman individualmente los
LSD de ambos operandos.

Se suman individualmente,
de derecha a izquierda, los
dígitos del mismo peso de
ambos operandos y el carry
del paso anterior

	Decimal	1 Binario	Octal	3 Hexa
Acarreo (Co)	00	1110 001	10	0
Sumando (A)	121	0111 1001	171	79
Sumando (B)	053	0011 0101	065	35
Resultado (S)	174	1010 1110	256	AE

$1+3=04$ $1+1=10$ $1+5=06$ $9+5=0E$

El acarreo o carry de dicha
suma se traslada al dígito
más significativo
inmediatamente a su
izquierda

Circuitos Sumadores

- Circuito semisumador (SS) o Half Adder (HA)

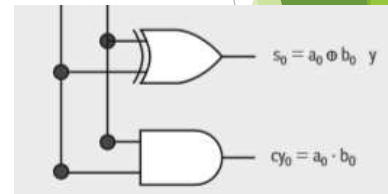
Es un circuito cuya función es sumar 2 bits sin tener en cuenta el acarreo anterior. Tiene dos salidas, una representa la suma y la otra, el valor del acarreo (carry).



donde:

a_0	a_0	a_1	a_1
+	+	+	+
0	1	0	1
0	1	1	0

a	b	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

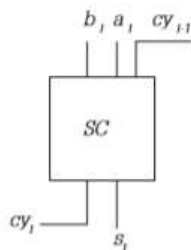


Circuitos Sumadores

- Circuito sumador completo (SC) o Full Adder (FA)

Este circuito tiene como finalidad sumar 2 bits y el acarreo anterior, generando como salidas el resultado de la suma y el nuevo acarreo.

Genéricamente:



donde:

a_i	a_i	a_i
+	+	+
0	0	0
0	1	1
a)	b)	

b_i = dígito i de la variable B a sumar
 a_i = dígito i de la variable A a sumar
 Cy_{i-1} = acarreo anterior
 S_i = Suma
 Cy_i = Acarreo actual

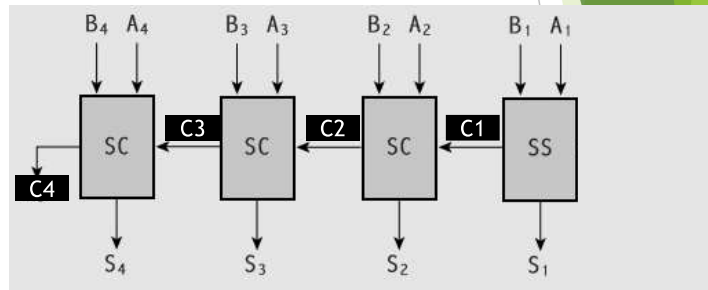
a	b	Cy_{i-1}	S	Cy_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Circuitos Sumadores

- Circuito Sumador Paralelo de 4 bits:

Ahora analizaremos cómo a partir de la expresión de una función booleana se puede confeccionar el circuito que lleve a cabo la operación que describe. Supongamos que queremos sumar dos números de 8 dígitos cada uno:

Cy	0	0	1	1	0	0	0	0
A	1	0	1	1	0	1	1	0
+ B	0	0	0	1	1	0	0	1
S	1	1	0	0	1	1	1	1



Sumador binario paralelo con acarreo serie

Circuito Sumador

La tabla de verdad muestra todas las posibles combinaciones de valores para un dígito binario de cada operando y para el carry.

Suma binaria				
Ci	A	B	Co	Suma
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

